SOFTWARE REQUIREMENTS SPECIFICATION

for

NETVANGUARD

(Network Intrusion Detection System & PCAP Security Analyzer)

Prepared by : Eman Fatima (Leader), Sarah Liaqat, Hadia

Submitted to : Sir Talha
Lecturer

October 30, 2026

Contents

1 Introduction **4**
- 1.1 Purpose . . . 4
- 1.2 Intended Audience and Reading Suggestions . . . 4
- 1.3 Project Scope . . . 4

2 Overall Description **6**
- 2.1 Product Perspective . . . 6
- 2.2 Architecture Diagram . . . 6
- 2.3 User Classes and Characteristics . . . 6
- 2.4 Product Functions . . . 6
- 2.5 Operating Environment . . . 8
- 2.6 Design Workflows . . . 8

3 System Features and Functional Requirements **9**
- 3.1 Description and Priority . . . 9
- 3.2 Detailed Functional Requirements . . . 9
  - 3.2.1 FR-1.1: PCAP Ingestion . . . 9
  - 3.2.2 FR-1.2: Multi-Layer Protocol Parsing . . . 10
  - 3.2.3 FR-2.1: IP Traffic Aggregation . . . 10
  - 3.2.4 FR-2.2: Top-Talkers Extraction . . . 11
  - 3.2.5 FR-2.3: Sliding Window Rate Tracking . . . 12
  - 3.2.6 FR-3.1: Port Scan Detection . . . 12
  - 3.2.7 FR-3.2: SYN Flood Detection . . . 13
  - 3.2.8 FR-3.3: Blocklist IP Lookup . . . 14
  - 3.2.9 FR-3.4: Ping Flood Detection . . . 14
  - 3.2.10 FR-4.1: Alert Formatting & Logging . . . 15
  - 3.2.11 FR-4.2: Interactive HTML Report Generation . . . 15
  - 3.2.12 FR-5.1: Rule Configuration Management . . . 16
- 3.3 Technology Stack & Project Management . . . 17
- 3.4 Team Roles & Task Allocation . . . 17
- 3.5 8-Week Development Roadmap . . . 17
- 3.6 GitHub Workflow & Governance . . . 18

4 Other Nonfunctional Requirements **19**
- 4.1 Performance Requirements . . . 19
- 4.2 Security Requirements . . . 19
- 4.3 Software Quality Attributes . . . 19

4.4 Business Rules 19
4.5 Backup and Recovery Requirements 20

5 Other Requirements **21**

1 Introduction

1.1 Purpose

In modern enterprise networks and organizational IT infrastructures, real-time visibility into network traffic and immediate detection of malicious behavioral anomalies are critical. Traditional packet analysis workflows rely on complex, heavy enterprise tools or manual packet inspection, which lack custom automated threat detection rules, lightweight sliding-window rate tracking, and integrated triage capabilities. "NetVanGuard" is designed as a high-performance, modular Network Intrusion Detection System (NIDS) and PCAP Security Analyzer. The software ingests network packet recordings (`.pcap` files) and live traffic streams, decomposes individual network frame headers, tracks host communications through advanced data structures, and raises real-time security alerts upon detecting anomalous or malicious behavior.

1.2 Intended Audience and Reading Suggestions

This Software Requirements Specification (SRS) is intended for system architects, cybersecurity engineers, software developers, quality assurance testers, and project evaluators. The document comprehensively details the functional and non-functional requirements, architectural design layers, data structure models, project roles, development roadmaps, and software quality constraints governing NetVanGuard.

1.3 Project Scope

NetVanGuard provides a multi-layer analytical framework structured around four core software modules: Packet Capture & Parsing, Statistical Aggregation, Security Detection Engine, and Interactive Reporting / User Interface.

Packet Capture & Parser: Ingests standard `.pcap` packet files (generated via Wireshark or `tcpdump`) or captures live interface traffic. It deconstructs multi-layer headers including IPv4/IPv6, TCP, UDP, and ICMP protocols to extract source/destination IP addresses, port numbers, flag states, payload sizes, and precise packet timestamps.

Statistical Aggregation Engine: Maintained through optimized Computer Science Data Structures (DSA). It computes traffic volume metrics, identifies high-bandwidth endpoints ("Top Talkers") using a max-heap/priority queue, tracks active TCP session handshakes, and records protocol distributions.

Security Detection Engine: Executes deterministic rule evaluations against live or parsed packet streams to catch network threats, including Port Scans, SYN Floods, Ping

Floods (ICMP overload), and communication with known malicious IP addresses via a high-performance Trie lookup structure.

Reporting & UI Console: Formats structured event logs, generates console dashboard summaries and interactive HTML report pages, and exposes real-time security alerts for rapid incident response.

2 Overall Description

2.1 Product Perspective

NetVanGuard functions as a high-performance, modular security application designed to bridge low-level network protocol parsing with real-time algorithm detection logic. Following a strict 5-tier architectural framework, the application separates hardware/packet ingestion, raw packet decoding, memory state management (DSA logic), security detection processing, and user presentation layers.

2.2 Architecture Diagram

The overall system architecture illustrating the 5-tier structure—Users/Clients, Presentation Layer, Application Layer with Control Gateway, Data Structure & Storage Layer, and External Integrations—is shown below:

2.3 User Classes and Characteristics

NetVanGuard accommodates 3 main user roles:

- **Security Analyst (Eman Fatima - Lead):** Performs threat hunting, analyzes PCAP traces, triages high-severity alert logs, and adjusts threshold settings.
- **Network Engineer (Sarah Liaqat):** Monitors protocol balances, identifies high-bandwidth endpoints, and inspects transport handshakes.
- **System Administrator (Hadia):** Manages system blocklists, configures user permissions, exports audit reports, and ensures pipeline stability.

2.4 Product Functions

NetGuard NIDS delivers complete traffic analysis and threat monitoring through several core operational sub-functions:
Multi-Layer Packet Decoding: Parses raw byte buffers to extract fields: `src_ip`, `dest_ip`, `src_port`, `dest_port`, `protocol_type` (TCP, UDP, ICMP), `flags` (SYN, ACK, FIN, RST), `payload_len`, and `timestamp`.
Traffic Statistics Aggregation: Tracks packet totals, byte counters, and protocol breakdown ratios using efficient Hash Maps (`ip_map`).
Top-Talkers Identification: Implements a Max-Heap / Priority Queue structure to

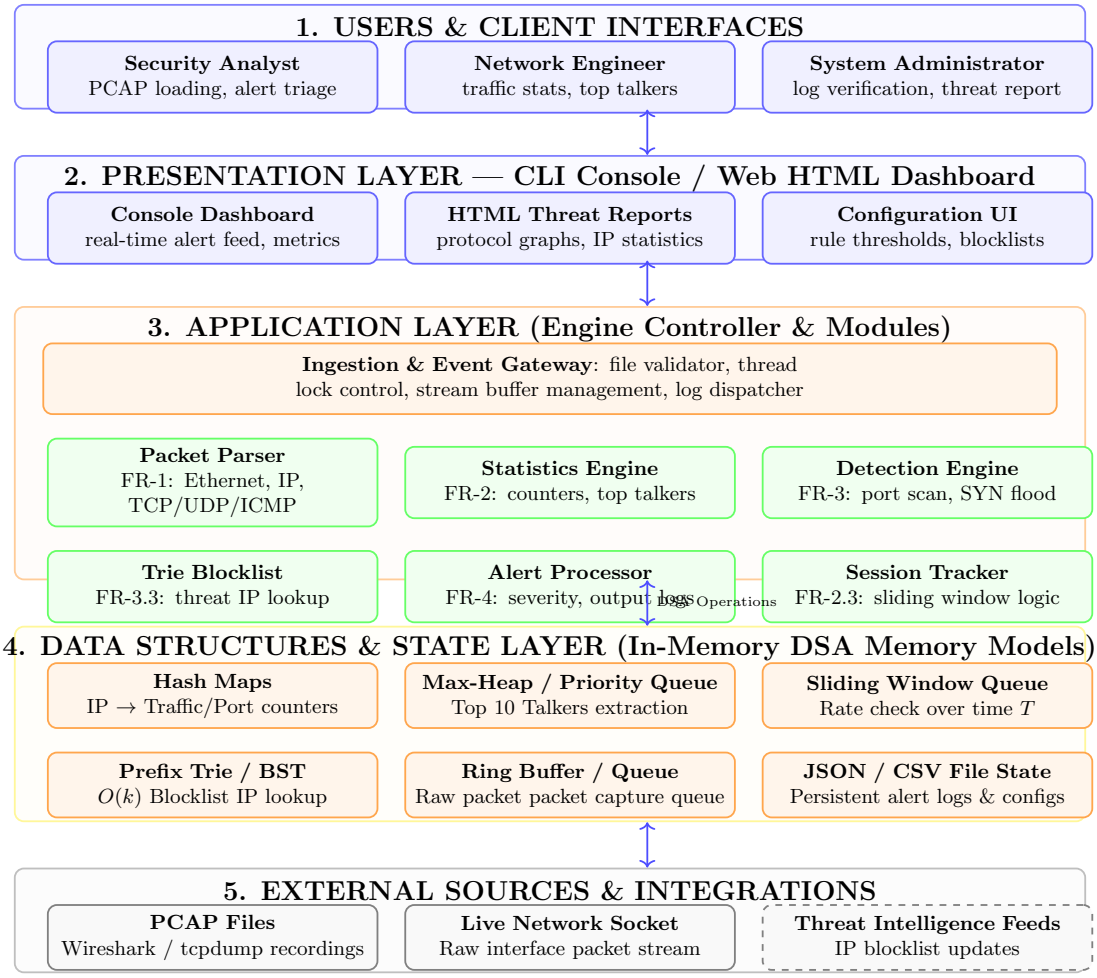


Figure 2.1: NetVanGuard System Architecture Diagram

dynamically extract the top N bandwidth-consuming IP addresses in $O(N \log k)$ time.
Attack Detection Suite: Applies algorithmic evaluation to flag malicious behavior including Port Scanning, SYN Flooding, Ping Floods, and Blocklisted IP interactions.
Blocklist Prefix Trie: Utilizes a Trie data structure to execute $O(k)$ prefix matching against thousands of known malicious IP subnets.
Alert Logging & HTML Reporting: Formats security triggers into structured JSON/CSV logs and compiles summary HTML dashboard documents.

2.5 Operating Environment

NetGuard NIDS is constructed in modular C++/Python with zero reliance on non-standard graphical frameworks. It runs across Linux (Ubuntu/Debian, CentOS), macOS, and Microsoft Windows environments. The application supports standard libpcap/WinPcap libraries and compiles under GCC 10+, Clang 12+, or MSVC 2019+.

2.6 Design Workflows

System operations follow three primary operational pipelines:

- **Packet Ingestion & Parsing Workflow:** Select `.pcap` File / Interface → Validate File Header → Extract Protocol Layers → Construct Packet Meta-Object.
- **Threat Analysis Workflow:** Pass Packet to Detection Engine → Check Blocklist Trie → Update Sliding Window Rate Counters → Evaluate Rule Criteria → Emit Security Alert.
- **Statistical Summary Workflow:** Update IP Hash Maps → Push Counters to Max-Heap → Rebalance Top-10 Talkers → Export Visual HTML Report.

3 System Features and Functional Requirements

3.1 Description and Priority

The major features of NetGuard NIDS are prioritized below:

1. **Packet Capture & Decoding Engine (High Priority):** Ingests `.pcap` recordings and decodes headers accurately across layer-3 and layer-4 protocols.
2. **Security Threat Detection Engine (High Priority):** Evaluates attack signatures including Port Scans, SYN Floods, Ping Floods, and Blocklist hits.
3. **High-Performance Data Structure Layer (High Priority):** Employs Hash Maps, Priority Queues, Sliding Windows, and IP Tries for real-time computational performance.
4. **Traffic Aggregation & Top Talkers (Medium Priority):** Aggregates total bandwidth usage per endpoint and protocol ratios.
5. **Reporting & Alert Dashboard (Medium Priority):** Outputs structured security log files and generates interactive HTML summary dashboards.

3.2 Detailed Functional Requirements

3.2.1 FR-1.1: PCAP Ingestion

Function
Parse and process standard `.pcap` traffic recordings.
Description
The system shall read `.pcap` formatted packet capture files and extract raw frame bytes for analysis.
Inputs
Valid `.pcap` file path from filesystem.
Source
Security Analyst via command-line arguments or file selection menu.
Outputs
Sequence of raw packet payload buffers and capture metadata.
Destination
Packet Decoder module.

Action
The system opens the specified `.pcap` file, verifies its magic number header, and iterates through individual packet frames.
Requirements
The input file must conform to standard libpcap or pcap-ng format specifications.
Pre-condition
Target `.pcap` file must exist and have valid read permissions.
Post-condition
All stored packet frames are streamed sequentially into the parsing queue.
Side Effects
Allocates memory buffers for packet byte streams.

3.2.2 FR-1.2: Multi-Layer Protocol Parsing

Function
Decode Ethernet, IP, TCP, UDP, and ICMP frame headers.
Description
The system shall unpack layer-2, layer-3, and layer-4 protocol fields from raw packet byte arrays.
Inputs
Raw packet byte array.
Source
PCAP Ingestion module.
Outputs
Structured `Packet` object containing extracted protocol header fields.
Destination
Statistics Engine and Security Detection Engine.
Action
The system reads byte offsets to extract source IP, destination IP, protocol ID, source port, destination port, TCP flag bits, and payload length.
Requirements
Malformed packet headers must be caught gracefully without crashing the application.
Pre-condition
Raw packet buffer must be non-empty.
Post-condition
A standardized internal packet object is instantiated.
Side Effects
Increments the total parsed packet counter.

3.2.3 FR-2.1: IP Traffic Aggregation

Function
Maintain traffic volume metrics per unique IP address.
Description

The system shall accumulate packet counts and aggregate byte volumes per source and destination IP using an in-memory Hash Map.

Inputs

Structured `Packet` object.

Source

Multi-Layer Protocol Parser.

Outputs

Updated Hash Map key-value entries (`IP Address` $\rightarrow$ `Metrics`).

Destination

Statistics State Repository.

Action

The system updates existing IP metric structures or inserts a new key into the hash map in $O(1)$ expected time.

Requirements

Hash collisions must be handled efficiently using chaining or open addressing.

Pre-condition

Packet object contains valid IPv4 or IPv6 address headers.

Post-condition

IP traffic counters reflect the latest packet transmission.

Side Effects

Expands in-memory hash table size as new endpoints are discovered.

3.2.4 FR-2.2: Top-Talkers Extraction

Function

Identify highest-bandwidth network endpoints using a Priority Queue.

Description

The system shall calculate and maintain the top N bandwidth-consuming IP addresses using a Max-Heap structure.

Inputs

Aggregated IP traffic metrics map.

Source

IP Traffic Aggregation module.

Outputs

Ordered list of top N IP addresses sorted by total byte volume.

Destination

Reporting Console and Dashboard view.

Action

The system inserts IP metric entries into a Max-Heap and extracts the top elements in $O(N \log k)$ operational time.

Requirements

Extraction operation must complete in under 50 milliseconds for up to 100,000 distinct IP records.

Pre-condition

IP traffic aggregation hash map contains at least one record.
Post-condition
Top talkers ranking list is refreshed for user display.
Side Effects
Re-balances the internal binary heap tree.

3.2.5 FR-2.3: Sliding Window Rate Tracking

Function
Track packet rate frequencies over time windows.
Description
The system shall maintain a sliding time window queue of timestamps to calculate instantaneous packet arrival rates per endpoint.
Inputs
Packet timestamp and source IP.
Source
Multi-Layer Protocol Parser.
Outputs
Current packet-per-second (PPS) rate for evaluated IP.
Destination
Security Detection Engine.
Action
The system pushes the current packet timestamp to an IP-specific queue and pops expired timestamps outside time window T.
Requirements
Time window evaluation duration T shall be configurable (e.g., 1.0 to 10.0 seconds).
Pre-condition
Packet contains precise arrival timestamp.
Post-condition
Old timestamps are purged, returning active rate count.
Side Effects
Frees stale timestamp elements from memory queues.

3.2.6 FR-3.1: Port Scan Detection

Function
Detect single-source port sweeping behavior.
Description
The system shall trigger a security alert if a single source IP touches 50 or more unique destination ports within a 3-second window.
Inputs
Source IP, destination port, and packet timestamp.
Source
Security Detection Engine.

Outputs
Port Scan Alert event containing target IP, unique port count, and time span.
Destination
Alert Logger and UI Console.
Action
The system tracks unique destination ports per source IP in a set within the sliding window, raising an alert if size threshold ≥ 50.
Requirements
Threshold settings must be configurable via system parameters.
Pre-condition
Packet protocol is identified as TCP or UDP.
Post-condition
A high-severity security alert entry is created upon threshold breach.
Side Effects
Locks source IP in active alert monitoring list.

3.2.7 FR-3.2: SYN Flood Detection

Function
Identify half-open TCP connection flood attacks.
Description
The system shall detect SYN flood attacks by identifying disproportionate ratios of TCP SYN packets lacking corresponding ACK responses.
Inputs
TCP flags (SYN, ACK), source IP, destination IP.
Source
Security Detection Engine.
Outputs
SYN Flood Alert event detailing target IP and unanswered SYN rate.
Destination
Alert Logger.
Action
The system increments SYN counters per IP pair and decrements upon receiving an ACK. If unacknowledged SYNs exceed 500 per second, an attack alert is generated.
Requirements
Connection tracking must accommodate out-of-order packet arrival.
Pre-condition
Packet protocol is identified as TCP.
Post-condition
SYN flood alert is dispatched to logging queues.
Side Effects
Updates active connection tracker state.

3.2.8 FR-3.3: Blocklist IP Lookup

Function
Match traffic against known malicious IP subnets using a Trie.
Description
The system shall compare every packet's source and destination IP against a malicious IP database stored in a Trie data structure.
Inputs
Source IP and Destination IP string/bits.
Source
Security Detection Engine.
Outputs
Blocklist Match Alert with threat classification level.
Destination
Alert Logger and Console UI.
Action
The system performs bitwise prefix traversal through the IP Trie, identifying matching malicious subnet nodes in $O(k)$ time ($k \leq 32$ for IPv4).
Requirements
Trie prefix lookup must complete in $O(1)$ relative to total rules loaded.
Pre-condition
Blocklist file must be pre-loaded into the IP Trie at startup.
Post-condition
Triggers immediate blocklist security event if match is positive.
Side Effects
None.

3.2.9 FR-3.4: Ping Flood Detection

Function
Detect ICMP request overload attacks.
Description
The system shall raise an alert if an endpoint receives an abnormal volume of ICMP Echo Request packets within a short duration.
Inputs
ICMP type/code, destination IP, timestamp.
Source
Security Detection Engine.
Outputs
Ping Flood Security Alert.
Destination
Alert Logger.
Action
The system monitors ICMP Type 8 (Echo Request) arrival rates per target IP, flagging

an alert if request frequency exceeds 200 packets/sec.
Requirements
Must distinguish between ICMP Echo Requests and Echo Replies.
Pre-condition
Packet protocol is identified as ICMP.
Post-condition
ICMP flood threat alert is generated.
Side Effects
Updates target IP ICMP rate tracker.

3.2.10 FR-4.1: Alert Formatting & Logging

Function
Format and persist security alerts to disk.
Description
The system shall format triggered alerts into structured JSON and CSV records and append them to disk logs.
Inputs
Alert object (Timestamp, Rule ID, Source IP, Dest IP, Severity, Message).
Source
Security Detection Engine.
Outputs
Structured log entries in `alerts.json` and `alerts.csv`.
Destination
Filesystem log directory.
Action
The system serializes the alert object into standardized text format and flushes the string to disk storage.
Requirements
Log writes must be thread-safe to prevent data corruption.
Pre-condition
Output log directory must be writeable.
Post-condition
Alert data is permanently stored on disk.
Side Effects
Appends new line entries to output file handles.

3.2.11 FR-4.2: Interactive HTML Report Generation

Function
Generate graphical summary HTML security reports.
Description
The system shall compile analysis results into a standalone HTML report containing protocol pie charts, top talker tables, and threat summary logs.

Inputs
Statistics map, Top Talker list, and Alert log repository.
Source
Reporting Engine.
Outputs
Formatted `report.html` file.
Destination
Output directory for user review.
Action
The system embeds metrics data into an HTML template structure with embedded styling and tables.
Requirements
Generated HTML must be self-contained and renderable offline in standard modern web browsers.
Pre-condition
Packet processing run must complete or be paused.
Post-condition
HTML report file is saved and ready for browser viewing.
Side Effects
Overwrites previous report file unless auto-timestamped.

3.2.12 FR-5.1: Rule Configuration Management

Function
Load and customize attack detection thresholds.
Description
The system shall parse an external configuration file (JSON/YAML) to set operational detection parameters and rate limits.
Inputs
Configuration file path.
Source
Security Analyst through command-line flag.
Outputs
Active configuration state structure.
Destination
Security Detection Engine.
Action
The system parses key-value pairs to set parameters such as `port_scan_threshold`, `syn_flood_pps`, and `blocklist_path`.
Requirements
Default values must be applied if configuration file is absent.
Pre-condition
Configuration file must be syntactically valid.
Post-condition

Detection thresholds are updated across engine modules.
Side Effects
Reconfigures internal detection rule parameters.

3.3 Technology Stack & Project Management

NetGuard NIDS is constructed using modern software engineering standards:
Core Programming Stack: C++17 / Python 3.10+ (for fast parsing and algorithmic performance).
Network Libraries: `libpcap` / `scapy` / Native socket APIs.
Data Structure Implementations: Standard Template Library (`std::unordered_map`, `std::priority_queue`), custom Trie prefix tree, and custom circular sliding window queue.
Build & Test Framework: CMake / Makefile, GoogleTest / pytest suite.

3.4 Team Roles & Task Allocation

The team responsibilities are explicitly structured as follows:

- **Eman Fatima (Member A - Leader):** Project management, Detection Engine rule logic (Port scan, SYN flood, Ping flood rules), sliding window queues, priority queue integration, and module integration.
- **Sarah Liaqat (Member B):** PCAP reader implementation, header parser (IP, TCP, UDP, ICMP), network socket handler, traffic aggregation hash map, and performance benchmarking.
- **Hadia (Member C):** Project skeleton design, logging infrastructure, blocklist Trie prefix structure, unit test suite, HTML/Console report generation, and SRS documentation.

3.5 8-Week Development Roadmap

The development lifecycle spans eight distinct phase milestones:

- **Weeks 1–2 (Planning & Architecture):** Finalize scope, author SRS document, set up GitHub repository rules, draft initial UML class and use-case diagrams, collect test `.pcap` samples.
- **Weeks 3–4 (Core Construction):** Build PCAP binary reader, implement protocol header parser, construct base project skeleton, set up unit test harnesses.
- **Weeks 5–6 (Detection Rules & DSA Layer):** Implement IP Hash Maps, Max-Heap for top talkers, Sliding Window rate checkers, Trie blocklist matcher, and core detection rules.

- **Week 7 (Integration & UI Reporting):** Integrate Parser, Detector, and Stats modules; implement console dashboard and HTML report builder; run performance tests on large PCAP captures (> 1 GB).
- **Week 8 (Testing, Delivery & Release):** Execute full integration test suites, resolve memory leak edge cases, complete system documentation, perform release tag cut (`v1.0.0`).

3.6 GitHub Workflow & Governance

To ensure code quality and collaborative integrity, the repository follows strict governance rules:

- **Branch Strategy:** `main` (stable release tag only), `dev` (active integration branch), and `feature/<task-name>` branches for individual tasks.
- **Pull Request (PR) Policy:** Direct pushes to `main` and `dev` are strictly blocked. Merges require at least one peer code review approval and passing test builds.
- **Issue Tracking:** All tasks are logged as GitHub Issues and tracked on an automated Projects Kanban board (`To Do`, `In Progress`, `Done`).
- **Sync Schedule:** Weekly 30-minute sync meetings led by Member A to perform code merges and assign upcoming milestone sprints.
- **Repository Layout:** Structured into `/src/parser`, `/src/detector`, `/src/stats`, `/src/report`, `/tests`, `/docs`, and `/samples`.

4 Other Nonfunctional Requirements

4.1 Performance Requirements

- The system shall process offline `.pcap` captures at a minimum throughput of 50,000 packets per second on standard multi-core processors.
- Blocklist IP lookup operations in the Trie structure shall complete in $O(k)$ time ($k \leq 32$ bits), independent of total blocklist entries.
- Memory usage shall remain bounded under 500 MB during processing of 1,000,000+ packet capture files.

4.2 Security Requirements

- The packet parsing engine must incorporate strict buffer bounds checking to prevent buffer overflow vulnerabilities when processing malicious or malformed packets.
- Output alert logs shall sanitize string inputs to prevent log injection or cross-site scripting (XSS) when rendered inside the HTML dashboard.

4.3 Software Quality Attributes

- **Reliability:** Exception handling mechanisms must isolate malformed packet errors without causing application termination or memory leakage.
- **Maintainability:** Codebase modules must adhere to SOLID design principles, maintaining clear decoupling between parsing, analysis, and UI layers.
- **Usability:** Clear command-line flags and readable visual report formats enable efficient operation for security personnel.

4.4 Business Rules

- Only packets with valid layer-3 IP checksums shall be processed for security rule analysis.
- Security alerts must include precise millisecond-level timestamps derived directly from original packet headers rather than host execution time.

4.5 Backup and Recovery Requirements

- In the event of an unhandled capture stream interruption, the application shall auto-flush pending in-memory alert buffers to disk before exiting.
- Configuration files shall be restored to safe factory defaults if an invalid JSON parameter structure is detected.

5 Other Requirements

Future enhancement phases (Phase 2) may incorporate real-time network interface binding via raw sockets, deep packet inspection (DPI) for application-layer protocols (HTTP, DNS, TLS SNI), dynamic machine-learning anomaly detection algorithms, and automated active response scripts (e.g., triggering `iptables` / firewall drop rules).